Data structure description language

The Data Structure Description Language (DSDL) is used to define data structures for exchange via the CAN bus. The DSDL definitions are used to automatically generate the message serialization/deserialization code for a certain programming language. The tool that generates source code from DSDL definition files is called the *DSDL compiler*.

File hierarchy

Each DSDL definition file specifies exactly one data structure that can be used for message broadcasting or a pair of structures that can be used for service invocation data exchange.

The DSDL source file must be named using the *data type name* and *default data type ID* (if needed) as shown below:

```
[default data type ID.]<data type name>.uavcan
```

A defined data structure must be contained in a namespace, which may in turn be *nested* within another namespace. A namespace that is not nested in another namespace is called a *root namespace*. For example, all standard data types are contained in the root namespace `uavcan`, which contains nested namespaces: `equipment`, `protocol`, etc.

The namespace hierarchy is mapped directly to the file-system directory structure, as shown in the example below:

```
+ uavcan                         <-- Root namespace
    + equipment                  <-- Nested namespace
        + ...
    + protocol                   <-- Nested namespace
        + 341.NodeStatus.uavcan <-- Definition of data type "uavcan.protocol.NodeStatus"
        + ...
    + Timestamp.uavcan           <-- Definition of data type "uavcan.Timestamp", default d
```

Notes:

- It is not necessary to explicitly define a default data type ID for non-standard data types (i.e., for vendor-specific or application-specific data types).
  - If the default data type ID is not defined by the DSDL definition, it will need to be assigned by the application at run time.
  - All standard data types have default data type ID values defined.
- Data type names are case sensitive, i.e., names `foo.Bar` and `foo.bar` are considered different. Names that differ only in case should be avoided, because it may cause problems on file systems that are not case-sensitive.
- Data types may contain nested data structures.
  - Some data structures may be designed for such nesting only, in which case they are not required to have a dedicated data type ID.
- *Full data type name* is a unique identifier of a data type constructed from the root namespace, all nested namespaces (if any), and the data type name itself, joined via the dot symbol (`.`), e.g., `uavcan.protocol.file.Read`.
  - The length of the Full data type name must not exceed 80 characters.

Specification

1. Introduction
2. Basic concepts
3. Data structure description language
4.1 CAN bus transport layer
4.2 UDP bus transport layer
4.3 MAVLink bus transport layer
4.4 CANFD bus transport layer
5. Application level conventions
6. Application level functions
7. List of standard data types
8. Hardware design recommendations
dsdl
  ardupilot
  uavcan

Contents

Data structure description language
  File hierarchy
    Service data structure
    Syntax
      Attribute definition
      Directives
      Comments
      Service response marker
    Primitive data types
    Naming rules
      Mandatory
      Optional
      Advisory
    Data type compatibility
      The concept of data type compatibility
      Signature
    Data type ID
    Data serialization
      Serialized data format

Service data structure

Since a service invocation consists of two network exchange operations, the DSDL definition for a service must define two structures:

- Request part - for request transfer (client to server).
- Response part - for response transfer (server to client).

Both request and response structures are contained within the same DSDL definition file, separated by a special statement (see the syntax chapter below).

Service invocation data structures cannot be nested.

Syntax

A data structure definition consists of *attributes* and *directives*. Any line of the definition file may contain at most one attribute definition or at most one directive. The same line cannot contain an attribute definition and a directive at the same time.

Lines should be separated with the ASCII line feed character (`\n` , code 10); vendor-specific data type definitions are allowed to use any other line ending convention.

An attribute can be either of the following:

- *Field* - a variable that can be modified by the application and exchanged via the network.
- *Constant* - an immutable value that does not participate in network exchange.

A directive is a statement that provides instructions to the DSDL compiler.

Aside from attributes and directives, a DSDL definition may contain the following entities:

- Comments
- Service response marker

The details are explained below.

A DSDL definition for a message data type may contain only the following:

- Attribute definitions (zero or more)
- Directives (zero or more)
- Comments (optional)

A DSDL definition for a service data type may contain only the following:

- Request part attribute definitions (zero or more)
- Response part attribute definitions (zero or more)
- Directives (zero or more)
- Comments (optional)
- Service response marker (exactly one)

Attribute definition

Field definition patterns:

1. `cast_mode field_type field_name`
2. `cast_mode field_type[X] field_name`
3. `cast_mode field_type[<X] field_name`

Standard data types
Vendor-specific data types

License

This work is licensed under a Creative Commons Attribution 4.0 International License.
Much of the content of this site is based upon prior work by Pavel Kirienko and the UAVCAN Development Team.

Constant definition pattern:

1. `cast_mode constant_type constant_name = constant_initializer`

Each component is discussed below.

Field type

The field type can be either a primitive data type (primitive data types are defined below) or a nested data structure.

A primitive data type can be referred simply by name, e.g., `float16`, `bool`.

A nested data structure can be referred by either of these two:

- Short name, e.g., `NodeStatus`, if both the referred and the referring data types are located in the same namespace. For example, it is possible to access `ns1.ns2.Type1` from `ns1.ns2.Type2` using a short name, but not from `ns1.ns2.ns3.Type3` or `ns1.ns4.Type4`.
- Full name, e.g., `uavcan.protocol.NodeStatus`. A full name allows access to the data type from any namespace.

A field type name can be appended with a statement in square brackets to define an array:

- Syntax `[X]` is used to define a static array of size exactly X items.
- Syntax `[<X]` is used to define a dynamic array of size from 0 to X-1 items, inclusively.
- Syntax `[<=X]` is used to define a dynamic array of size from 0 to X items, inclusively.

In the array definition statements above, X must be a valid integer literal according to the rules defined in the section dedicated to constant definitions.

Arrays of maximum size with less than one item are not allowed. Multidimensional arrays are not allowed.

Field name and constant name

For a message data type, all attributes must have a unique name within the data type.

For a service data type, all attributes must have a unique name within the same part (request/response) of the data type. In other words, service-type attributes can have the same name as long as they are separated by the service response marker.

Cast mode

Cast mode defines the rules of conversion from the native value of a certain programming language to the serialized field value. Cast mode may be left undefined, in which case the default will be used. Possible cast modes are defined below.

- `saturated` - This is the default cast mode, which will be used if the attribute definition does not specify the cast mode explicitly. For integers, it prevents an integer overflow - for example, attempting to write 0x44 to a 4-bit field will result in a bitfield value of 0x0F. For floating point values, it prevents overflow when casting to a lower precision floating point representation - for example, 65536.0 will be converted to a `float16` as 65504.0; infinity will be preserved.

- `truncated` - For integers, it discards the excess most significant bits - for example, attempting to write 0x44 to a 4-bit field will produce 0x04. For floating point values, overflow during downcasting will produce an infinity.

allowed as constant types).

A constant must be assigned with a constant initializer, which must be one of the following:

- Integer zero (0).
- Integer literal in base 10, starting with a non-zero character. E.g., 123, -12.
- Integer literal in base 16 prefixed with `0x`. E.g., 0x123, -0x12, +0x123.
- Integer literal in base 2 prefixed with `0b`. E.g., 0b1101, -0b101101, +0b101101.
- Integer literal in base 8 prefixed with `0o`. E.g., 0o123, -0o777, +0o777.
- Floating point literal. Fractional part with an optional exponent part, e.g., `15.75`, `1.575E1`, `1575e-2`, `-2.5e-3`, `+25E-4`. Not-a-number (NaN), positive infinity, and negative infinity are intentionally not supported in order to maximize cross-platform compatibility.
- Boolean `true` or `false`.
- Single ASCII character, ASCII escape sequence, or ASCII hex literal in single quotes. E.g., `'a'`, `'\x61'`, `'\n'`.

The DSDL compiler must implicitly convert the type of an initializer expression to the constant type if the target type can allocate the value with no data loss. If a data loss occurs (e.g., integer overflow, floating point number decays to infinity), the DSDL compiler should refuse to compile such data type.

Note that constants do not affect the serialized data layout as they are never exchanged via the network.

Void type

Void type is a special field type that is intended for data alignment purposes. The specification defines 64 distinct void types as follows:

- `void1` - 1 padding bit;
- `void2` - 2 padding bits;
- ...
- `void63` - 63 padding bits;
- `void64` - 64 padding bits.

A field of type void does not have a name and its cast mode cannot be specified. During message serialization, all void fields must be populated with zero bits; during deserialization, contents of the void fields should be ignored.

Directives

A directive is a single case-sensitive word starting with an “at sign” (`@`), possibly followed by space-separated arguments:

- `@directive`
- `@directive arg1 arg2`

All valid directives are documented below.

Union

Keyword: `@union`.

This directive instructs the DSDL compiler that the current message or the current part of a service data type (request or response) is a tagged union. A tagged union is a data structure that may encode either of its fields at a time. Such a data structure contains one

This directive must be placed before the first attribute definition.

The encoding rules for tagged unions are defined in the following chapters.

Comments

A DSDL description may contain comments starting from a number sign `#` up to the end of the line. Comments must be ignored by the DSDL compiler.

Service response marker

A service response marker separates the request and response parts of a service data type definition. The marker consists of three minus symbols (`-`) in a row on a dedicated line:

```
---
```

Primitive data types

These types are assumed to be built-in. They can be directly referenced from any data type of any namespace. The DSDL compiler should implement these types using the native types of the target programming language. Example mapping to native types is given for C/C++.

There is also a set of special built-in types `voidX`, where `X` is the number of bits, 1 to 64 inclusive; e.g. `void7`. They are used for data alignment purposes.

Name	Bit length	Possible representation in C/C++	Value range	Binary representation
bool	1	`bool` *(can be optimized for bit arrays)*	{0, 1}	One bit
int**X**	2 ≤ **X** ≤ 64	`int8_t`, `int16_t`, `int32_t`, `int64_t`	$[-(2^{X})/2, 2^{X}/2 - 1]$	Two's complement
uint**X**	2 ≤ **X** ≤ 64	`uint8_t`, `uint16_t`, `uint32_t`, `uint64_t`	$[0, 2^{X} - 1]$	
float16	16	`float`	±65504	IEEE754 binary16
float32	32	`float`	Approx. $\pm 10^{39}$	IEEE754 binary32
float64	64	`double`	Approx. $\pm 10^{308}$	IEEE754 binary64
void**X**	1 ≤ **X** ≤ 64	N/A	N/A	**X** zero bits (set to zero when encoding, ignore when decoding)

Naming rules

Mandatory

Field names, constant names, and type names must contain only ASCII alphanumeric characters and underscores (`[A-Za-z0-9_]`), and must begin with an ASCII alphabetic character (`[A-Za-z]`). Violation of this rule must be detected by the DSDL compiler and treated as a fatal error.

Optional

The following rules are recommended only (DSDL compilers are not required to enforce them):

- Field and namespace names should be all-lowercase words separated with underscores, and may include numbers, (e.g.: `field_name`, `my_namespace_7`).
- Constant names should be all-uppercase words separated with underscores, and may include numbers (e.g.: `CONSTANT_NAME`).
- Data type names should be in camel case (first letter of all words in uppercase) and may include numbers (e.g.: `TypeName`, `TypeName2`).

Advisory

The following rules should be considered by the application designer, but should not be enforced:

- Message names should be nouns or adjectives; service names should be verbs.
- The name of a message that carries a command should end with the word “Command”; the name of a message that carries state information should end with the word “Status”.
- The name of a service that is designed to obtain or to store data should begin with the word “Get” or “Set”, respectively.

Data type compatibility

The concept of data type compatibility

It is vital that all nodes exchanging some particular data structure use compatible DSDL definitions of it. Different DSDL definitions are considered compatible if they share the aspects listed below.

Binary layout

Encoded binary representation must be interpreted by different nodes in the same way. This implies that compatible data structures must have the same field types in the same order.

First definition:

```
uint8 a
uint8 b
```

Second definition:

```
uint12 a
uint4 b
```

Even though the bit length of the data structures above is the same, the binary layout is clearly not compatible.

Field name and order

Encoded binary representation must be interpreted by different nodes in the same way. This implies that compatible data structures must have the same field types and names in

First definition:

```
uint8 a
uint8 b
```

Second definition:

```
uint8 b
uint8 a
```

Even though the first and the second definitions share the same binary layout (two fields of type `uint8`), they feature different field names and therefore are semantically incompatible.

Constant definitions

Despite the fact that different constant values may render the different representations of the same data structure semantically incompatible, DroneCAN does not consider constant definitions in signature computation, because fixing constant definitions renders data types not extensible, i.e., any modification of a constant would break backward compatibility.

Signature

Incompatible data types may break the data exchange and cause unpredictable system behavior. In order to ensure type compatibility the concept of a *data type signature* is introduced. This section explains both the data type signature and a number of associated/auxiliary concepts.

Signature *usage* is discussed in the topic: CAN bus transport layer.

Signature hash function

The data type signature is based on CRC-64-WE:

- Name: CRC-64-WE
- Description: http://reveng.sourceforge.net/crc-catalogue/17plus.htm#crc.cat-bits.64
- Initial value: 0xFFFFFFFFFFFFFFFF
- Poly: 0x42F0E1EBA9EA3693
- Reverse: no
- Output XOR: 0xFFFFFFFFFFFFFFFF
- Check: 0x62EC59E3F1A4F00A

Source code in Python:

```
# License: CC0, no copyright reserved
class Signature:
    MASK64 = 0xFFFFFFFFFFFFFFFF
    POLY = 0x42F0E1EBA9EA3693

    def __init__(self, extend_from=None):
        if extend_from is not None:
            self._crc = (int(extend_from) & Signature.MASK64) ^ Signature.MASK64
        else:
            self._crc = Signature.MASK64

    def add(self, data_bytes):
        if isinstance(data_bytes, str):
            data_bytes = map(ord, data_bytes)
        for b in data_bytes:
            self._crc ^= (b << 56) & Signature.MASK64
```

```
                else:
                    self._crc <<= 1

    def get_value(self):
        return (self._crc & Signature.MASK64) ^ Signature.MASK64
```

Normalized data type definition

Some traits of a data type are important to ensure compatibility and some are not. A normalized data type definition is a definition that does not have such unimportant traits.

To obtain a normalized definition of a given data type, the following actions must be performed on its definition:

1. Remove comments.
2. Remove all constant definitions.
3. Ensure that all cast specifiers are explicitly defined; if not, add default cast specifiers.
4. For dynamic arrays, replace the max length specifier in the form `[<X]` to the form `[<=Y]`.
5. For nested data structures, replace all short names with full names.
6. Remove unimportant whitespace (empty lines, leading whitespace, trailing whitespace, more than one whitespace between tokens).
7. Prepend the DSDL definition with the full data type name on a separate line.
8. Replace newline characters with the ASCII line-feed character (code: 0x0A; escape sequence: `\n`).

Example for message type `A` in the namespace `root`:

```
#
# A header comment.
# Note that the formatting is broken deliberately.
#

@union

float16 foo

float16 BAR    = 12.34  # This is BAR
truncated uint8    bar

int32  FOO =   - 42
```

Normalized definition:

```
root.A
@union
saturated float16 foo
truncated uint8 bar
```

Example for service type `A` in the namespace `root`:

```
#
# A header comment.
# Note that the formatting is broken deliberately.
#

B foobar
float16 foo
float16 BAR    = 12.34  # This is BAR
```

```
int32  BAR =   -42
root.ns1.B baz
```

Normalized definition:

```
root.A
root.B foobar
saturated float16 foo
---
truncated uint8 foo
root.ns1.B baz
```

DSDL signature

DSDL signature is the product of the application of the *signature hash function* to a *normalized data type definition*.

It is important to understand that DSDL signature is not the same as data type signature (because the data type may reference definitions stored in several files).

Data type signature

Data type signature is a 64-bit integer value that is *guaranteed to be equal for compatible data types*.

Hence, it is said that *data types are compatible if their names and signatures are equal*.

The following is the data type signature computation algorithm for a given data type, where *hash* is the signature hash function described earlier:

1. Initialize the hash value with the DSDL signature of the given type.
2. Starting from the top of the DSDL definition, do the following for each nested data structure:
    1. Extend the current hash value with the data type signature of the nested data structure.
3. The resulting hash value will be the data type signature.

The hash extension algorithm:

1. Save the current hash value.
2. Feed the value the hash needs to be extended with to the hash function, byte by byte, LSB first.
3. Feed the saved hash value to the hash function, byte by byte, LSB first.

The data type signature computation algorithm has the following properties:

- Data type signature and DSDL signature of the same type are equal if the data type does not contain nested data structures.
- Data type signature is guaranteed to match only if all nested data structures are compatible.

The implementation of the algorithm explained above can be found in one of the existing implementations.

Aggregate signature

Aggregate data type signature is a signature computed for a set of data types.

Two sets of data types **X** and **Y** are compatible if each data type in **X** has exactly one compatible data type in **Y** and both sets are of equal size.

One possible way to ensure compatibility between **X** and **Y** is to directly check the compatibility of each element from **X** against each element from **Y**. DroneCAN, however, defines a computationally less expensive algorithm based on data type signature extension (as defined above).

The algorithm that computes the aggregate signature for the set of data types **A** is defined as follows, where *hash* is the signature hash function described earlier:

1. Sort **A** by full data type name lexicographically in descending order (encoding is ASCII, order is A to Z).
2. Initialize the hash value with the data type signature of the first data type from **A**.
3. For each data type **a** from **A**, starting from the second, do the following:
4. Extend the current hash value with the data type signature of **a**.

Then, *two sets of data types are considered to be compatible if their aggregate signatures are equal*.

Data type ID

The set of possible data type ID values is limited, so devices from different vendors may occasionally reuse the same data type ID for different purposes. A part of the data type ID space is reserved for standard data types. Since all DroneCAN nodes should share the same configuration for standard data types, collisions are unlikely to occur in that range. Another part of the data type ID space is dedicated for vendor-specific (or application-specific) IDs, and this range of IDs is always collision prone.

In order to allow the nodes from different vendors to be used in the same application, *the end user must be able to change the ID of any non-standard data type on each node*. Vendors are advised to provide the end user with an option to change the ID of any standard message type as well.

Notes:

- The reserved ID ranges listed in the Application level conventions.
- Message types and service types do not share the same set of possible data ID values (i.e., a message type and a service type can share the same data type ID with no conflict). You can learn more about this in the CAN bus transport layer specification.
- Changing ID does not affect data type compatibility.

Data serialization

Serialized data format

Serialized data are an ordered sequence of bit fields; no header or footer is provided, and fields are not implicitly aligned. DSDL authors are advised (but not required) to manually align fields at byte boundaries using the void data type, in order to simplify data layouts and improve the performance of serialization and deserialization routines.

The rules of encoding of different data types are defined below.

Primitive types

Type	Bit length	Binary representation
int**X**	**X**	Two’s complement signed integer

bool	1	Single bit
float16	16	IEEE754 binary16
float32	32	IEEE754 binary32
float64	64	IEEE754 binary64
void**X**	**X**	**X** zero bits; ignore when decoding

Nested data structures

Nested data structures are encoded in much the same way as if they were standalone top-level data structures, with one exception for *tail array optimization*, as defined below.

Fixed size arrays

Fixed-size arrays are encoded as a plain sequence of items, with each item encoded independently in place, with no alignment. No extra data is added.

Essentially, a fixed-size array of size **X** will be encoded exactly in the same way as a sequence of **X** fields of the same type in a row. Hence, the following data type:

```
AnyType[3] array
```

will have the same binary layout as:

```
AnyType array_0
AnyType array_1
AnyType array_2
```

The only difference is the representation in the target programming language.

Dynamic arrays

Dynamic array encoding rules are sophisticated; hence it is recommended to review the existing implementations for a deeper understanding.

There is no difference between the following two array definitions other than text representation for the human benefit:

```
AnyType[<42]  a # Max size 41
AnyType[<=41] b # Max size 41
```

Both forms are equivalent in terms of the data serialization.

Array items can be of variable length (for example, if the item's type is a data structure that itself contains dynamic arrays). As a result, array maximum and minimum size may not not always be the same.

Normally, a dynamic array will be encoded as a sequence of encoded items, prepended with an unsigned integer field representing the number of contained items - the *length field*. The bit width of the length field is a function of the maximum number of items in the array: `⌈log2(X + 1)⌉`, where X is the maximum number of items in the array. For example, if the maximum number of items is 251, the length field bit width must be 8 bits, or if the maximum number of items is 1, the length field bit width will be just a single bit.

The transport layer provides a data length for every received data transfer (with an 8-bit resolution); thus, in some cases, the array length information would be redundant as it can be inferred from the overall transfer length reported by the transport layer. Elimination of

1. The minimum bit length of an item type is not less than 8 bits - because the transport layer reports the transfer length with an 8-bit resolution.
2. The array is the last field in the top-level data structure - because, otherwise, a much more complicated logic would be required to derive the length.

The second condition means that tail array optimization would be possible only if there are no other fields in the output bit stream after the dynamic array. Only one array can be tail optimized. Below are some examples that illustrate when a tail array optimization is possible and when it is not.

```
# Type root.A
uint8 foo
# Tail array optimization is possible here
# It will be encoded just like a fixed-size array
# The number of elements will be inferred from the transfer length
uint8[<9] array

# Type root.B
float16 foo
# Tail array optimization will not be possible because item length is less than 8 bits
uint7[<=8] array

# Type root.C
# Tail array optimization will not be possible because the array is not the last field
uint8[<=8] array
float16 bar

# Type root.D
# Tail array optimization will not be possible because item length is less than 8 bits
bool[<=42] array

# Type root.E
# Tail array optimization will not be possible because the minimum item length is zero
root.D[<=42] array     # Refer to root.D definition above
```

The same rules are applied to more sophisticated cases, for example when multiple nested types with dynamic arrays are used. Such structure may produce very complex binary layouts, but the same array encoding rules are still applicable.

```
# Type root.Z
# Refer to root.A definition above
# This array will be tail optimized
# Arrays inside root.A will not be tail optimized
root.A[<=2] array

# Type root.Y
# This array will NOT be tail optimized
# Arrays inside root.A will NOT be tail optimized
root.A[<=2] array
# The last field here effectively disables any arrays from being tail optimized
float16 baz

# Type root.Q
int4 fooz
# Just an array that can be tail optimized, no nested types
float64[<=64] array

# Type root.X
# This array will NOT be tail optimized
# Because the minimum bit length of root.Q is 4 bits (less than 8 bits)
# However, the last array in the last item will be tail optimized
root.Q[<=12] array
```

Unions

Unions are encoded as two subsequent entities:

1. The union tag;
2. The selected field.

The union tag is an unsigned integer, whose bit length is a function of the number of fields in the union: `⌈log2(N)⌉`, where N is the number of fields in the union. The value encoded in the union tag is the index of the selected field. Field indexes are assigned according to the order in which they are defined, starting from zero; i.e. the first defined field gets index 0, second defined field gets index 1, and so on.

Consider the following example:

```
#
# This is a union.
#

@union                  # In this case, the union tag requires 2 bits

uint16 FOO = 42         # A regular constant attribute

uint16 a                # Index 0
uint8 b                 # Index 1
float64 c               # Index 2

uint32 BAR = 42         # Another regular constant
```

In order to encode the value `b`, which, according to the definition, has the data type `uint8`, the union tag should be assigned the value 1. The following structure will have identical layout:

```
uint2 tag
uint8 b
```

By way of an example, if the value of `b` was 7, the resulting encoded byte sequence would be (in binary):

```
01000001 11000000
```

Byte order and bit order

Byte order: least significant byte (LSB) first, also known as little-endian.

Bit order: bits are filled from the most significant to the least significant, i.e., the most significant bit has index 0.

The resulting bit sequence must be aligned to 1 byte; pad bits must be set to zero.

For the purpose of example, the following data will be encoded according to the defined order:

1. 0xbeda of bit length 12
2. -1 of bit length 3
3. -5 of bit length 4
4. -1 of bit length 2

The resulting byte sequence is shown on the following diagram:

Scalar fields serialization example

	Byte index	0								1								2								3							
	Bit index	0	1	2	3	4	5	6	7	0	1	2	3	4	5	6	7	0	1	2	3	4	5	6	7	0	1	2	3	4	5	6	7
	Bit position	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
	Encoded bit values	**1**	**1**	**0**	**1**	**1**	**0**	**1**	**0**	**1**	**1**	**1**	**0**	**1**	**1**	**1**	**1**	**0**	**1**	**1**	**1**	**1**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**
Value index	**Value to encode**	**Target bit length**	**Binary representation before truncation**	4 most significant bits are truncated; the value is then converted to little endian representation																													
0	0xBEDA	12	1011'1110 1101'1010																														
1	-1	3	111	Two's complement																													
2	-5	4	1011	Two's complement																													
3	-1	2	11	Two's complement																													
4	0x88	4	1000'1000	4 most significant bits are truncated																													
N/A	Alignment	N/A	All zero	The encoded message must be byte aligned																													

We recommend you review the existing implementations.

Standard data types

The DSDL definitions of the standard data structures are available in the DSDL repository, which is linked here, and in a later section of the specification.

Information concerning development and maintenance of the standard DSDL definitions is available here.

Vendor-specific data types

Vendors must define their specific data types in a separate namespace, which should typically be named to match their company name. Separation of the vendor's definitions into a dedicated namespace ensures that no name conflicts will occur in systems that utilize vendor-specific data types from different providers. Note that, according to the naming requirements, the name of a DSDL namespace must start with an alphabetic character; therefore, a company whose name starts with a digit will have to resort to a mangled name, e.g. by moving the digits towards the end of the name, or by spelling the digits in English (e.g. 42 - `fortytwo`).

Defining vendor-specific data types within the standard namespace `uavcan` is explicitly prohibited. The standard namespace will always be used only for standard definitions.

Generally speaking it is desirable for "generic" data types to be included into the standard set. Vendors should strive to design their data types as generic and as independent of their specific use cases as possible. The SI system of measurement units should be preferred, as data type definitions that make unnecessary deviations from SI will not be accepted into the standard set.